

JUSTIFICACIÓN DE LA ELECCIÓN DE BOUML

BOUML es una herramienta de modelado UML 2.0 de código abierto, desarrollada por Bruno Pagès y publicada bajo GPLv2 [1].

Características Principales:

- Código Abierto y Gratuito
- Multiplataforma (Windows, Linux, Mac OS X)
- Ultra-ligero y rápido
- Soporte UML 2.0
- Generación de código en C++, Java, Python, PHP, IDL, SQL
- Ingeniería Inversa
- Extensible mediante API

Comparación con Otras Herramientas:

Característica	BOUML	StarUML	Visual Paradigm
Licencia	Gratuito (GPL)	Freemium	De pago
Rendimiento	Muy Alto	Bajo	Medio
Lenguajes Generados	C++, Java, Python, PHP, SQL	Java, C#, Python	Java, C#, PHP, Python
Colaboración	No	No	Sí

Por qué BOUML para este Trabajo:

La selección de BOUML se justifica por: accesibilidad y coste cero, enfoque en MDD, velocidad y eficiencia, soporte para C++ (lenguaje nativo), y facilidad de demostración práctica.

MANUAL DE USO DE BOUML (PASO A PASO)

Instalación:

1. Navegar a <http://bouml.free.fr/> y descargar la versión para tu sistema operativo.
2. Ejecutar el instalador correspondiente.
3. Al iniciar BOUML por primera vez, configurar el "Own Identifier" (ej: 1).

Creación del Proyecto:

1. Menú Project → New
2. Seleccionar directorio y nombrar proyecto (ej: SGCVIA-MDD)
3. Crear Package: clic derecho → New → Package (ej: ModeloAtenciones)

Creación del Diagrama de Clases:

1. Clic derecho sobre el package → New → Class view (ej: VistaSGCVIA)
2. Clic derecho sobre la Class view → New → Class diagram (ej: DiagramaAtenciones)

Creación de Clases y Atributos:

1. En la paleta, seleccionar "Class" y hacer clic en el diagrama
2. Nombrar la clase (ej: Usuario)
3. Doble clic en la clase → pestaña "Attributes" → New
4. Configurar: Name, Type, Visibility, Multiplicity

Creación de Métodos:

1. En el editor de la clase, ir a pestaña "Operations"
2. New → configurar Name, Return type, Visibility
3. Agregar parámetros en la sección "Parameters"

Creación de Relaciones:

1. **Herencia:** Botón "Generalization" → de clase hija a clase padre
2. **Asociación:** Botón "Association" → entre dos clases
3. **Composición:** Association → marcar "Composition" en propiedades
4. **Agregación:** Association → marcar "Aggregation" en propiedades

Configuración de Cardinalidades:

1. Doble clic en la línea de asociación
2. Pestaña "Role" → Multiplicity: 1, 0.., 1.., 0..1

Creación de Enums:

1. Crear clase con estereotipo "enum" marcado
2. Agregar literales como atributos

Guardar el Proyecto:

- File → Save All o Ctrl+Shift+S

Configurar Lenguaje de Generación:

- Menú Languages → seleccionar C++ (marcar con check)

Configurar Directorio de Generación:

1. Clic derecho en proyecto → Edit Generation Settings...
2. Pestaña "Directory" → establecer ruta de salida

Crear Artefactos para Generación:

1. Crear Deployment View: New → Deployment view (ej: DeploySGCVIA)
2. Asociar Class View con Deployment View
3. En cada clase: clic derecho → Create associated artifact

Generar el Código:

1. Clic derecho en Deployment View → Generate → C++

REFERENCIAS BIBLIOGRÁFICAS

[1] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25-31, Feb. 2006.